

CHAPTER - 1

ABOUT THE COMPANY

- **Company Name:** Crismatech Automation Pvt. Ltd.
- **Location:** Bangalore, Karnataka
- **Established:** 2018/2019
- **Domain:** IT, Automation & Software Services
- **Specialization:** DevOps, Cloud Computing, AI, IoT, Software Development.

1.1 Corporate Introduction

Crismatech Automation Pvt. Ltd. is a Bangalore-based IT firm that has emerged as a forward-thinking leader in the Silicon Valley of India. Established around 2019, the company focuses on bridging the gap between academic theory and practical industry application through high-end software solutions and internship programs. As a VTU-registered entity, it aligns with global standards to provide hands-on digital transformation services.

1.2 Company Profile

The organization offers a diverse portfolio of services, including:

Core Technologies: Artificial Intelligence (AI), Internet of Things (IoT), and Robotic Process Automation (RPA).

- **Specialized Services:** Web design, mobile application development, and Electric Vehicle (EV) technology.
- **Key Projects:** The company handles diverse initiatives such as Solid Waste Management eco-systems (Eco-Sys), Medical Billing Data Entry, and the development of Electric Autonomous Vehicles.
- **Mission:** To empower businesses through innovation and meticulously crafted user-centric ecosystems.

CHAPTER - 2

ACTIVITIES PERFORMED

Our three-month journey at Crismatech was a roller-coaster of technical breakthroughs, frustrating roadblocks, and eventual triumphs. The learning curve was steep, often feeling more like a vertical wall, but the experience gained was invaluable.

Month 1: The Docker Deep Dive and Scripting Struggles The internship began with a reality check. We thought we knew Linux, but we quickly realised our knowledge was superficial. The first major learning experience involved shell scripting. We were tasked with automating the setup of a development environment. We spent days wrestling with Bash scripts, learning the hard way about exit codes, permissions, and environment variables. We learned that a script that runs on Ubuntu might fail on CentOS, teaching us the pain of OS fragmentation.

Then came **Docker**. We had used Docker before, but only to pull existing images. Our task was to containerise a complex Python micro service application. We struggled initially with Dockerfiles. We created images that were 1GB in size, which our mentor laughed at (kindly). This experience taught us about multi-stage builds. We learned to use a heavy image for compiling dependencies and then copy the artifacts to a tiny Alpine Linux image for the final product, reducing the size to 100MB. We also faced the "Networking Nightmare." We tried to get two containers to talk to each other using localhost, which failed miserably. This forced us to learn about Docker Bridges and how container networking actually isolates processes. It was a week of frustration that ended in a "Eureka!" moment when the database container finally accepted a connection from the app container.

Month 2: The Jenkins Pipeline and the "Works on My Machine" Syndrome The second month was dominated by **Jenkins**. Setting up a CI/CD pipeline sounds easy in tutorials, but in a corporate proxy environment, it is a beast. We spent three days just trying to get Jenkins to talk to our Git repository. We learned about SSH keys, webhooks, and firewall rules—networking fundamentals we had neglected in college.

The most vivid learning experience here was the failure of our first automated build. The code worked perfectly on our laptops but crashed inside the Jenkins pipeline. We spent hours debugging, blaming the server. Eventually, we realised we had hardcoded a file path that only existed on our

Windows machines, while the server was Linux. This was a harsh lesson in cross-platform compatibility and the importance of relative paths.

We also learned about **Secret Management**. In our initial attempts, we pushed a file containing AWS API keys to a public Git repository. Our mentor immediately flagged this, explaining the catastrophic security implications. We had to scrub the commit history—a terrifying process—and learn to use Vaults and environment variables injection. This experience instilled in us a security-first mindset that no lecture could have provided.

Month 3: Kubernetes, Chaos, and Monitoring The final month was the most intense. Moving from Docker Compose to **Kubernetes (K8s)** felt like moving from driving a car to flying a spaceship. Concepts like Pods, Deployments, ReplicaSets, and Ingresses were overwhelming. We spent days staring at YAML files, trying to understand why our pods were in a CrashLoopBackOff state. We learned to read logs effectively—not just looking at the last line, but tracing the stack trace to the root cause.

The implementation of the **Self-Healing Simulator** was the highlight. We wrote a Python script to intentionally stress the CPU. Watching the Grafana dashboards light up in red was terrifying. However, seeing the Kubernetes Horizontal Pod Autoscaler (HPA) kick in and spawn five new pods to handle the load was magical. We learned that "reliability" isn't about preventing failure; it's about managing it. We learned how to write queries in PromQL (Prometheus Query Language) to create custom alerts.

The interpersonal learning was just as significant. Working as a trio, we had disagreements. One of us wanted to use AWS, another wanted Azure. One wanted to write scripts in Python, another in Bash. We learned to compromise based on technical merit rather than personal preference. We learned that "perfect is the enemy of good"—sometimes you have to ship a working MVP (Minimum Viable Product) rather than waiting for the perfect solution. These soft-skill experiences—negotiation, documentation, and time management—were the glue that held our technical skills together.

CHAPTER - 3

TASKS PERFORMED

- Implementing CI/CD Pipelines
 - Monitoring and Logging with Grafana
 - Chaos Engineering (Monkey-style scripts)
 - Security Auditing (SAST, SonarQube, and CVE scanning)
 - Automated Quality Assurance (E2E Testing)
-
- **Implementing CI/CD Pipelines:** Developed fully automated pipelines that eliminate slow, error-prone manual deployments, ensuring rapid and reliable software delivery.
 - **Monitoring and Logging:** Integrated **Grafana** and Prometheus to gain real-time visibility into system health, allowing for proactive performance tuning.
 - **Chaos Engineering:** Executed "Monkey-style" scripts to randomly stress test systems, ensuring the infrastructure could stay resilient and fail-safe under unexpected conditions.
 - **Security Auditing (Shift-Left Security):** Integrated **SonarQube** for Static Application Security Testing (SAST) and scanned third-party libraries for known CVEs (Common Vulnerabilities and Exposures) early in the development cycle.
 - **Automated Quality Assurance:** Developed End-to-End (E2E) testing suites to validate user flows like login and signup, ensuring that new code did not break existing functionality.

CHAPTER - 4

SPECIFIC OUTCOMES REFLECTION NOTES

By the conclusion of the ZenithPipeline project, our transformation from students to junior DevOps engineers was marked by distinct, measurable learning outcomes. These outcomes span technical competencies, architectural understanding, and professional methodology.

Advanced Technical Competency in the DevOps Toolchain We have achieved proficiency in the "Infinity Loop" of DevOps tools.

Source Control Management (SCM): We mastered Git beyond the basics. We can now manage complex merge conflicts, use git rebase to keep history clean, and utilize tags for release management.

Containerization: We can confidently write multi-stage Dockerfiles, manage volumes for data persistence, and use Docker Compose for local orchestration. We understand the difference between a container and a VM at the kernel level (namespaces and cgroups).

Orchestration: We have a working knowledge of Kubernetes. We can deploy applications using Helm charts, manage secrets using K8s secrets, and configure Ingress controllers for external access.

CI/CD Automation: We can design Jenkins pipelines using Groovy syntax (Jenkinsfile). We know how to set up build stages, post-build actions, and integrate notification systems (like Slack or Email) to alert the team of build failures.

Monitoring & Logging: We moved from "guessing" to "knowing." We can set up Prometheus scrapers to collect metrics and design Grafana dashboards that provide real-time visibility into CPU, Memory, and Network I/O.

Understanding of High Availability and Scalability Architecture We no longer view software as a static binary running on a single computer. We understand architecture.

Load Balancing: We learned how Nginx and HAProxy distribute traffic to prevent any single server from being overwhelmed.

Stateless vs. Stateful: We grasped the crucial difference between stateless applications (which can be easily scaled) and stateful applications (databases) that require careful handling of data persistence.

Self-Healing Principles: We internalized the concept that systems should repair themselves. We understand how to define "liveness" (is the app running?) and "readiness" (can the app take traffic?) probes to ensure zero-downtime deployments.

Infrastructure as Code (IaC) and Configuration Management We have adopted the outcome that infrastructure is software.

Reproducibility: We learned that manual server setup is a technical debt. By using Ansible playbooks, we ensured that every server is configured identically, eliminating "configuration drift."

Version Control for Infra: We learned that infrastructure changes should be committed to Git, reviewed, and audited just like application code. This provides a safety net, allowing us to "rollback" infrastructure changes if they cause issues.

The Shift-Left Security Mindset (DevSecOps) A major outcome was the integration of security into the development phase.

Static Application Security Testing (SAST): We learned to use SonarQube to catch code vulnerabilities (like SQL injection risks) *before* the code is even built.

Dependency Scanning: We learned that vulnerabilities often come from third-party libraries. We used tools to scan our Docker images for known CVEs (Common Vulnerabilities and Exposures).

Professional Engineering Practices

Agile Fluency: We are now comfortable working in Sprints. We understand the value of a "Stand-up" to unblock teammates and the importance of a "Retrospective" to improve processes.

Documentation Discipline: We realised that code without documentation is a black box. We developed the habit of writing README.md files, documenting API endpoints, and commenting on complex logic.

Resilience and Debugging: Perhaps the most important outcome is a changed attitude toward errors. We no longer panic when we see a red error message. We see it as a clue. We have developed a systematic approach to debugging: isolate variables, check logs, reproduce the issue, and apply the fix.

CHAPTER - 5

CONCLUSION

The three-month internship at Crismatech Automation Private Limited has been a defining chapter in our academic and professional lives. The project, **ZenithPipeline: Automated CI/CD & Self-Healing Infrastructure Simulator**, served as the perfect vehicle to traverse the vast and complex landscape of DevOps Engineering.

Summary of Achievement We began as a group of three students with theoretical knowledge of programming but limited exposure to operations. We ended as a cohesive engineering unit capable of deploying, managing, and scaling cloud-native applications. We successfully built a pipeline that automates the drudgery of software deployment, reducing the release time from hours to minutes. More importantly, we built a safety net—a self-healing infrastructure that can withstand the chaos of real-world failures.

The simulator we built proved that "uptime" is not a happy accident; it is an engineered outcome. By simulating server crashes and traffic spikes, we demonstrated that with the right tools (Kubernetes, Prometheus) and the right configuration, a system can be resilient. We proved that automation is the key to reliability.

Reflection on the Industry This experience highlighted the rapid evolution of the software industry. It became clear that the role of a "Developer" and an "Operator" is merging. The future belongs to those who can write the code *and* understand the ecosystem in which it lives. We observed how critical "Time to Market" is for businesses, and how DevOps is the accelerator that allows companies to innovate faster without breaking things.

Future Scope While ZenithPipeline is a functional success, it is only a start. In the future, this project could be expanded to include "GitOps" using tools like ArgoCD, where the Git repository acts as the single source of truth for the entire cluster state. We could also implement "Service Mesh" (like Istio) for even more granular control over traffic and security.

Final Words In conclusion, this internship was not just about learning tools like Docker or Jenkins; it was about adopting a culture. A culture of collaboration, automation, and continuous improvement. We leave Crismatech with a profound appreciation for the invisible engineering that keeps the internet running. We are grateful for the mentorship, the challenges, and the failures that

forced us to grow. We are now prepared to enter the workforce not just as coders, but as problem solvers and DevOps engineers, ready to build the resilient systems of tomorrow.